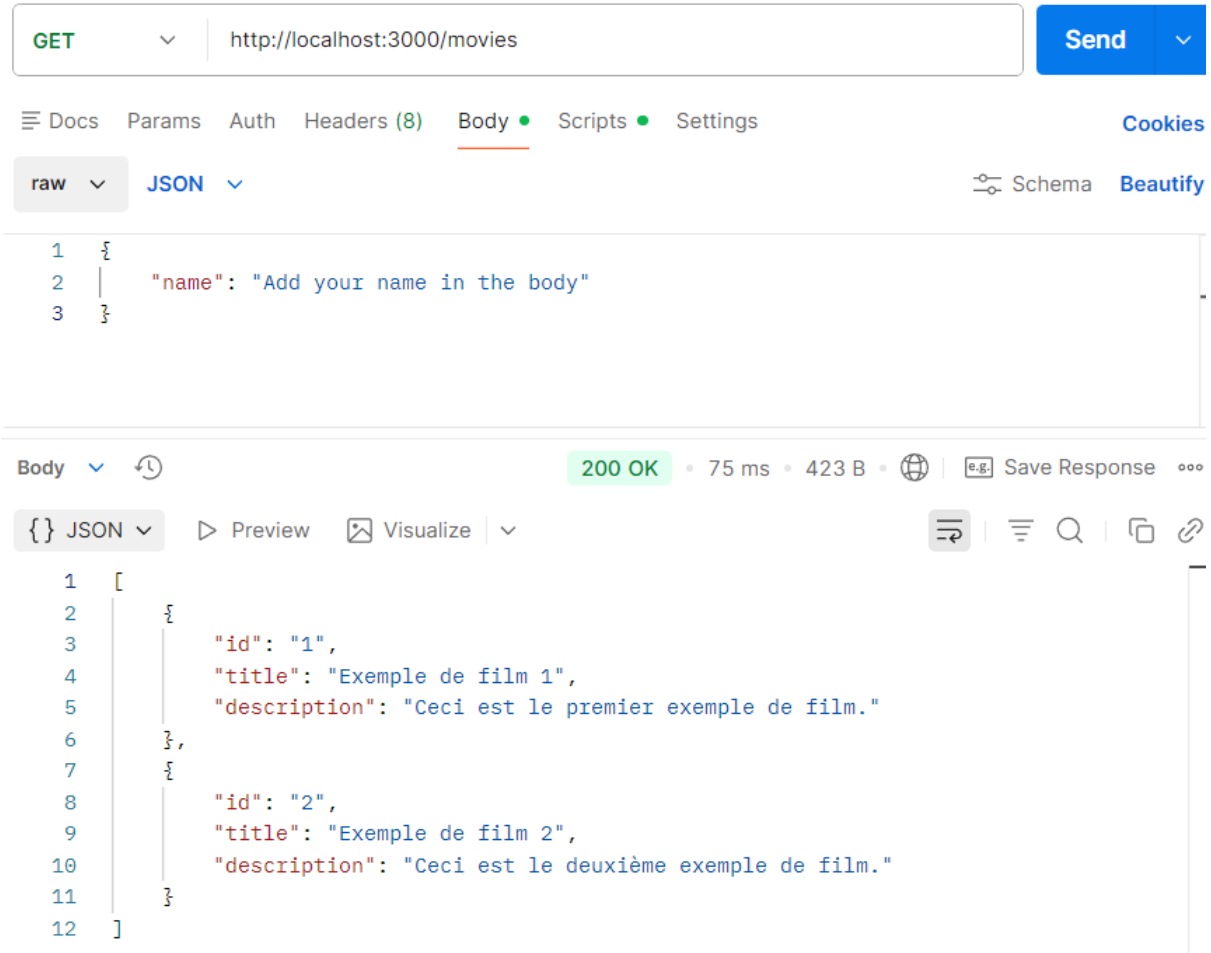
TP8 : Microservices avec REST, GraphQL et gRPC

Aya cherchir

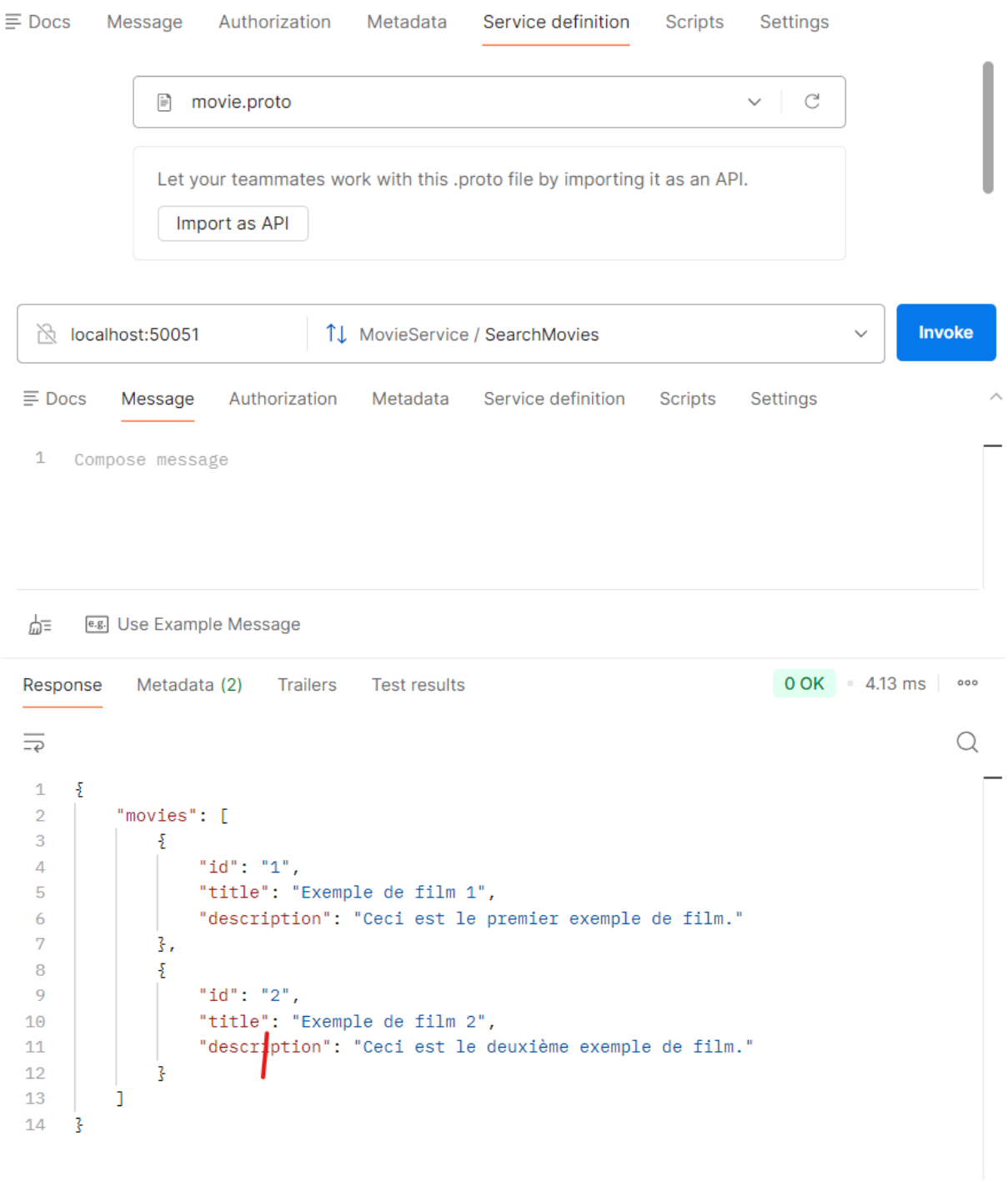
11. Démarrer les microservices et le gateway en respectant l'ordre suivant :
    1. node movieMicroservice.js
    2. node tvShowMicroservice.js
    3. node apiGateway.js

12. Tester l'API Gateway avec REST et GraphQL

REST (via Postman)



GraphQL (via Postman) pour movies



13. Ajouter des opérations de création, de mise à jour et d'effacements de film et de série TV sur REST, GraphQL et gRPC.

1-REST dans apiGateway.js, j'ajoute ces methodes (pour movies)

```
app.post("/movies", async (req, res) => {
  const movie = await movieService.createMovie(req.body);
  res.json(movie);
});
app.put("/movies/:id", async (req, res) => {
  const movie = await movieService.updateMovie(req.params.id, req.body);
  res.json(movie);
});
app.delete("/movies/:id", async (req, res) => {
  const result = await movieService.deleteMovie(req.params.id);
  res.json(result);
});
```

j'ajoute ces methodes (pour tvShows):

```
app.post("/tvshows", async (req, res) => {
  const tv = await tvService.createTvShow(req.body);
  res.json(tv);
});
app.put("/tvshows/:id", async (req, res) => {
  const tv = await tvService.updateTvShow(req.params.id, req.body);
  res.json(tv);
});
app.delete("/tvshows/:id", async (req, res) => {
  const result = await tvService.deleteTvShow(req.params.id);
  res.json(result);
});
```

2-GraphQL: ajout de mutation dans schema.gql:

```
type Mutation {

addMovie(title: String!, description: String!): Movie
updateMovie(id: String!, title: String, description: String): Movie
deleteMovie(id: String!): Boolean


addTVShow(title: String!, description: String!): TVShow
updateTVShow(id: String!, title: String, description: String): TVShow
deleteTVShow(id: String!): Boolean
}
```

modification de resolvers.js:

```
const resolvers = {
  Mutation: {
    addMovie: (_, args) => movieService.createMovie(args),
    updateMovie: (_, args) => movieService.updateMovie(args.id, args),
    deleteMovie: (_, { id }) => movieService.deleteMovie(id),

    addTVShow: (_, args) => tvService.createTvShow(args),
    updateTVShow: (_, args) => tvService.updateTvShow(args.id, args),
    deleteTVShow: (_, { id }) => tvService.deleteTvShow(id),
  }
};
const movieProto = grpc.loadPackageDefinition(movieProtoDefinition).movie;
const tvShowProto = grpc.loadPackageDefinition(tvShowProtoDefinition).tvShow;
// Définir les résolveurs pour les requêtes GraphQL
const resolvers = {
```

3-gRPC (Proto + Implementation)
ajout nouveaux services dans movie.proto

```
// Définition du service de films
service MovieService {
rpc GetMovie(GetMovieRequest) returns (GetMovieResponse);
rpc SearchMovies(SearchMoviesRequest) returns (SearchMoviesResponse);
// Ajouter d'autres méthodes au besoin
rpc CreateMovie (Movie) returns (Movie);
rpc UpdateMovie (Movie) returns (Movie);
rpc DeleteMovie (MovieId) returns (Response);
}
```

et dans tvShow.proto

```
service TVShowService {
rpc GetTvshow(GetTVShowRequest) returns (GetTVShowResponse);
rpc SearchTvshows(SearchTVShowsRequest) returns (SearchTVShowsResponse);
// Ajouter d'autres méthodes au besoin
rpc CreateTvShow (TvShow) returns (TvShow);
rpc UpdateTvShow (TvShow) returns (TvShow);
rpc DeleteTvShow (TvShowId) returns (Response);
}
```

implémentation de nouveaux services dans movieMicroService.js

```
CreateMovie: (call, callback) => {
  const movie = call.request;
  movies.push(movie);
  callback(null, movie);
},
UpdateMovie: (call, callback) => {
  const updated = call.request;
  // update logic ici
  callback(null, updated);
},

DeleteMovie: (call, callback) => {
  // delete logic ici
  callback(null, { success: true });
},
```

et dans tvShowMicroService.js

```
CreateTvShow: (call, callback) => {
  const tv_show = call.request;
  tv_shows.push(tv_show);
  callback(null, tv_show);
},

UpdateTvShow: (call, callback) => {
  const updated = call.request;
  // update logic ici
  callback(null, updated);
},

DeleteTvShow: (call, callback) => {
  // delete logic ici
  callback(null, { success: true });
}
```

14. Modifier l'application pour se connecter à la base de données de votre choix en respectant le schéma de données.

1-installation de mongoosh

```
PS C:\Users\asus\tp-microservices> npm install mongoose

37 packages are looking for funding
  run `npm fund` for details

1 moderate severity vulnerability

To address all issues, run:
  npm audit fix
```

2-creation fichier db.js pour connecter à MongoDB

```
const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    await mongoose.connect("mongodb://localhost:27017/microservicesDB");
    console.log("MongoDB connected");
  } catch (err) {
    console.error(err);
    process.exit(1);
  }
};

module.exports = connectDB;
```

appeler dans chaque microservice

```
// movieMicroservice.js
const connectDB = require("./db");

connectDB();
```

creation eux models Movie.js et TvShow.js(devenir collection)

```
const mongoose = require("mongoose");

const movieSchema = new mongoose.Schema({
  title: String,
  description: String
});

module.exports = mongoose.model("Movie", movieSchema);
```

```
const mongoose = require("mongoose");

const tvShowSchema = new mongoose.Schema({
  title: String,
  description: String
});

module.exports = mongoose.model("TVShow", tvShowSchema);
```